

# Supplementary Material 2

## Main Model

Juan Sebastián Felipe Olmos Núñez  
Fernando Fagundes Ferreira

Cristina Adams  
Vitor Sanches

2026-06-22

## Table of contents

|          |                     |          |
|----------|---------------------|----------|
| <b>1</b> | <b>Introduction</b> | <b>1</b> |
| <b>2</b> | <b>Model</b>        | <b>2</b> |
| <b>3</b> | <b>Results</b>      | <b>5</b> |
| <b>4</b> | <b>Key points</b>   | <b>9</b> |

## List of Figures

|   |                                                                                                                                          |   |
|---|------------------------------------------------------------------------------------------------------------------------------------------|---|
| 1 | System richness for all configurations . . . . .                                                                                         | 5 |
| 2 | Clan richness measures for all configurations . . . . .                                                                                  | 5 |
| 3 | Community richness measures for all configurations . . . . .                                                                             | 6 |
| 4 | Comparison diversity metrics community and clan level . . . . .                                                                          | 7 |
| 5 | Comparison diversity metrics community and clan y for parallel and cross for 2 and 3<br>communitites. Alternative visuaization . . . . . | 8 |

## 1 Introduction

This supplementary material presents additional results for the main model presented in the paper “Agrobiodiversity dynamics in traditional socioecological systems: an agent based model approach on the role of kinship systems and variety transmission”. The main results and discussions are presented in the paper. Therefore, here we present the model’s implementation and additional results. The code is available in the GitHub repository.

## 2 Model

The model was implemented in python and the code is available in the repository, inside the folder local-kin-transmission-model. We created a function for initialization and a function for post-marital transmission system and kinship system.

Community structured initialization:

```
def init_bench_sum(N=20, n=1, v_total=30, initial_var=9, seed=None):
    """
    v_total is the TOTAL number of varieties across the system.
    Initialization for N farms and n communities.
    Each community gets v_total // n varieties, always exclusive to that community.
    """
    if seed is not None:
        random.seed(seed)

    varieties_per_com = v_total // n

    matriz = [[[0] for _ in range(v_total)] for _ in range(N * n)]

    for i in range(N * n):
        community_idx = i // N
        offset = community_idx * varieties_per_com

        for _ in range(initial_var):
            pos = offset + random.choice(range(varieties_per_com))
            matriz[i][pos][0] += 1

    return matriz
```

Simulation for cross transmission endogamy:

```
def simul_cross_100_EN(matriz, steps=100, n=1):
    data = []

    div_dict = div_com_clan_gamma_alfa_beta(matriz, n=n, clans=False)
    jaccard_dict = jaccard_clan_com(matriz, n=n, clans=False)
    data.append({
        'time' : 0,
        'matriz': np.array(matriz).copy(),
        **div_dict,
        **jaccard_dict
    })

    for t in range(1, steps + 1):
        daughters = [random.randint(1, 3) for _ in range(len(matriz))]
        sons = [random.randint(1, 3) for _ in range(len(matriz))]
```

```

farm_order = list(range(len(matriz)))
random.shuffle(farm_order)

for farm in farm_order:
    origin = [i for i, spot in enumerate(matriz[farm])
               if spot[0] > 0]

    candidates = [i for i in range(len(matriz))
                   if i != farm and sons[i] > 0]
    n_others = min(daughters[farm], len(candidates))

    if n_others == 0:
        for _ in range(3):
            origin = [i for i, spot in enumerate(matriz[farm])
                       if spot[0] > 0]
            pos_remove = random.choice(origin)
            matriz[farm][pos_remove][0] -= 1

            origin = [i for i, spot in enumerate(matriz[farm])
                       if spot[0] > 0]
            pos_add = random.choice(origin)
            matriz[farm][pos_add][0] += 1
        continue

    others = random.sample(candidates, n_others)

    for other_farm in others:
        sons[other_farm] -= 1

        for _ in range(3):
            origin = [i for i, spot in enumerate(matriz[farm])
                       if spot[0] > 0]
            receiver = [i for i, spot in enumerate(matriz[other_farm])
                        if spot[0] > 0]

            pos_delete = random.choice(receiver)
            pos_pass = random.choice(origin)

            matriz[other_farm][pos_delete][0] -= 1
            matriz[other_farm][pos_pass][0] += 1

if t % 1 == 0:
    div_dict = div_com_clan_gamma_alfa_beta(matriz, n=n, clans=False)
    jaccard_dict = jaccard_clan_com(matriz, n=n, clans=False)
    data.append({
        'time' : t,
        'matriz': np.array(matriz).copy(),

```

```

        **div_dict,
        **jaccard_dict
    })

    return data, matriz

```

Two additional functions were created:

- Function to run various simulations of a specific number of steps

```

def various_simul(simul_func, N=20,
                  v_total=30, steps=30,
                  simulations=100, n=1, com_init = True):
    all_data = []
    for s in range(simulations):
        if com_init:
            matriz = init_bench_sum(N=N, v_total=v_total, n=n)
        else:
            matriz = init_bench_sum_clan(N=N, v_total=v_total, n=n)
        data, _ = simul_func(matriz, steps=steps, n=n)
        for record in data:
            record['simulation'] = s
            record['n'] = n
            record['func'] = simul_func.__name__
            record['init'] = 'com' if com_init else 'clan'
            all_data.append(record)
    return pd.DataFrame(all_data)

```

- Function to run various simulations with different parameters

```

def exp_com_init_distrib():
    communities = [1,2,3]
    functions = [simul_cross_100_EN,
                 simul_cross_66_EN, simul_cross_33_EN, simul_paralel_EN,
                 simul_cross_100_DU, simul_cross_66_DU, simul_cross_33_DU, simul_paralel_DU]
    all_dfs = []
    for funct in functions:
        for n in communities:
            df = various_simul(funct, n=n, simulations=100, steps=150)
            all_dfs.append(df)

    df_final_original_com = pd.concat(all_dfs, ignore_index=True)
    df_final_original_com.to_pickle(save_path)

```

### 3 Results

Here we present results for both initialization configurations. The data used to render the results available in this report (if you are seeing it from Quarto) should be downloaded from this link or created running the code.

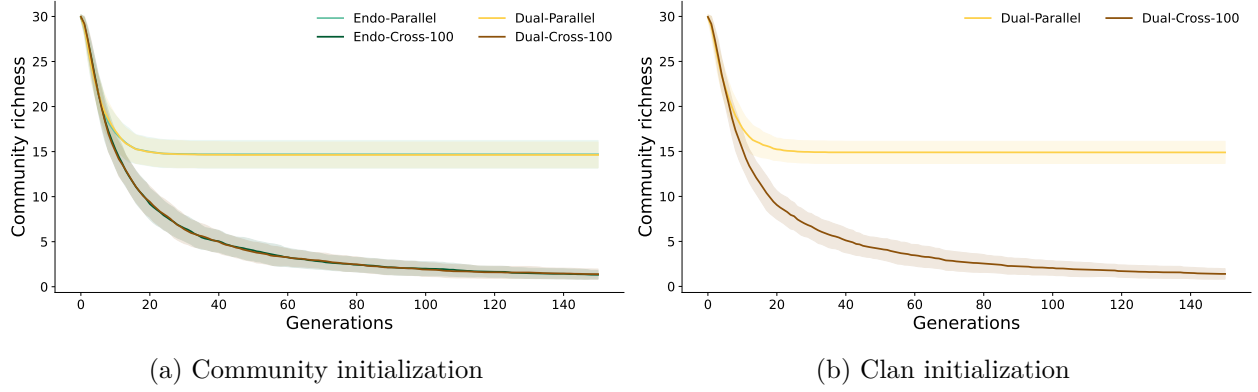


Figure 1: System richness for all configurations

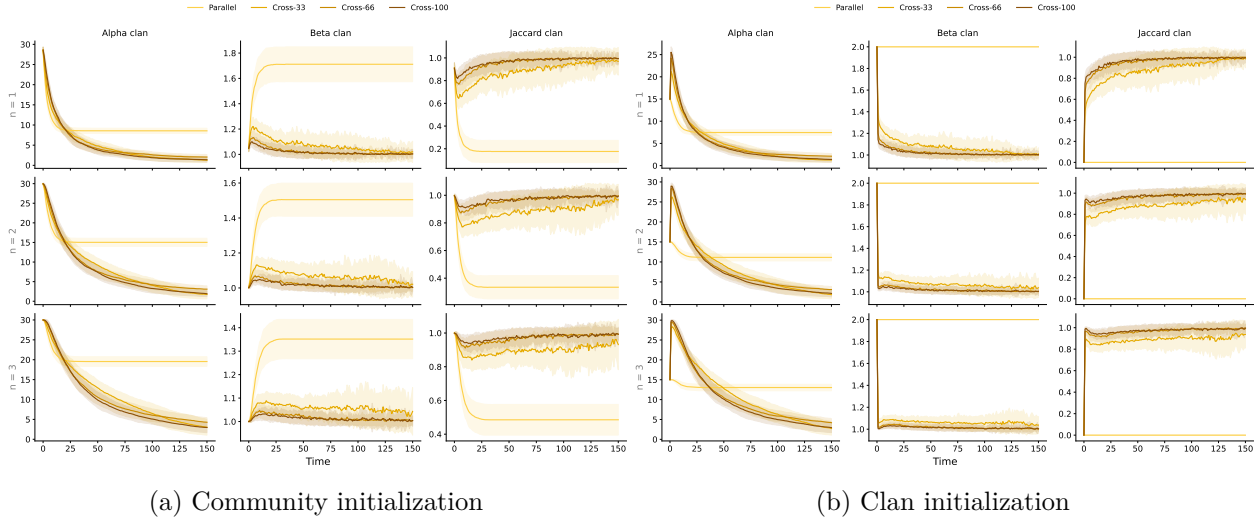


Figure 2: Clan richness measures for all configurations

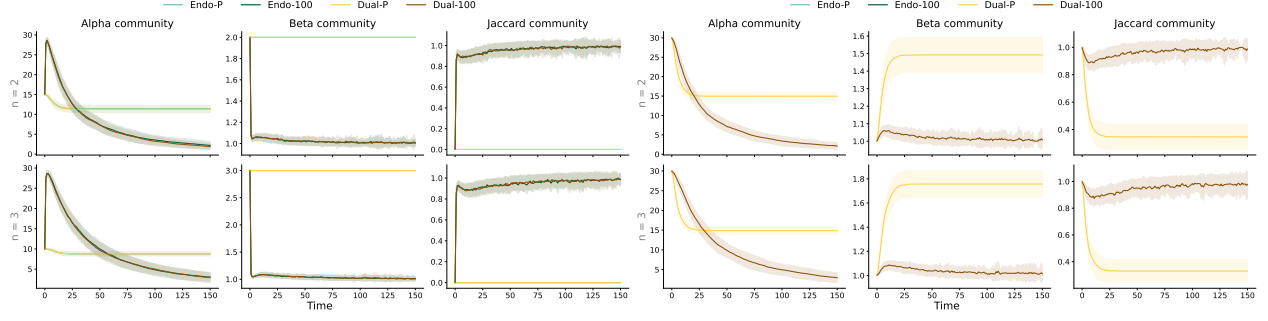
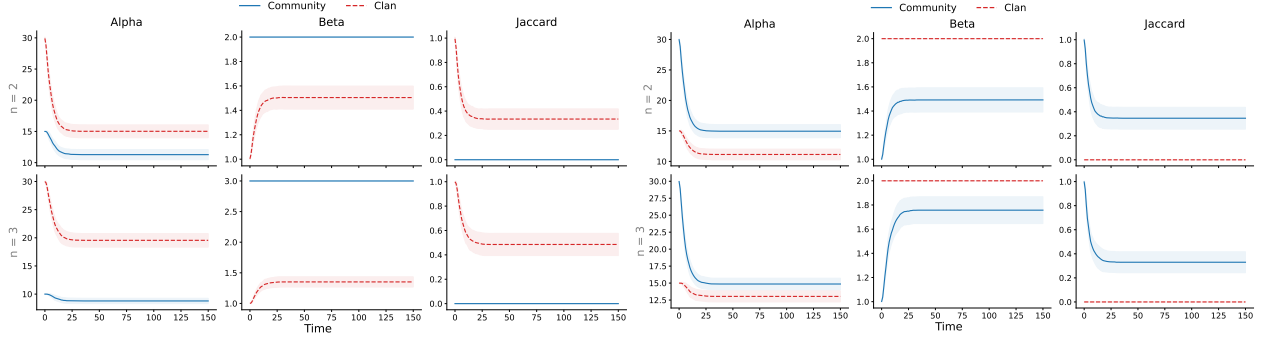
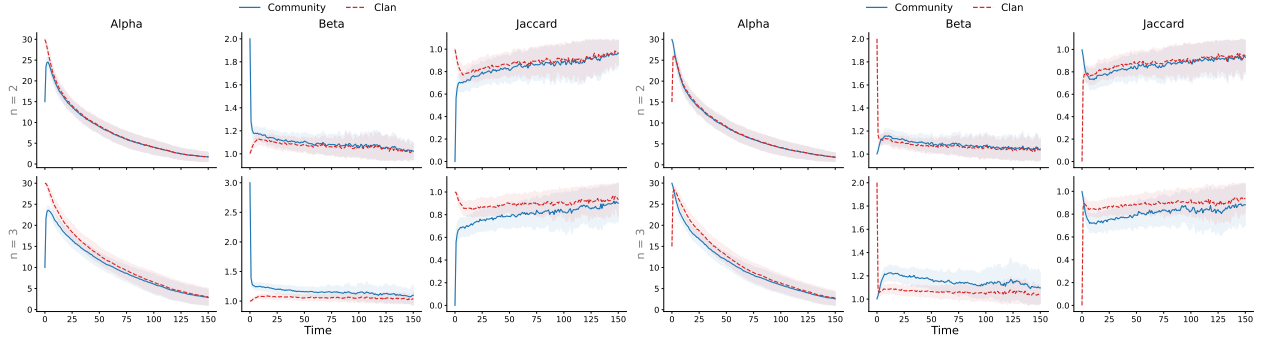


Figure 3: Community richness measures for all configurations



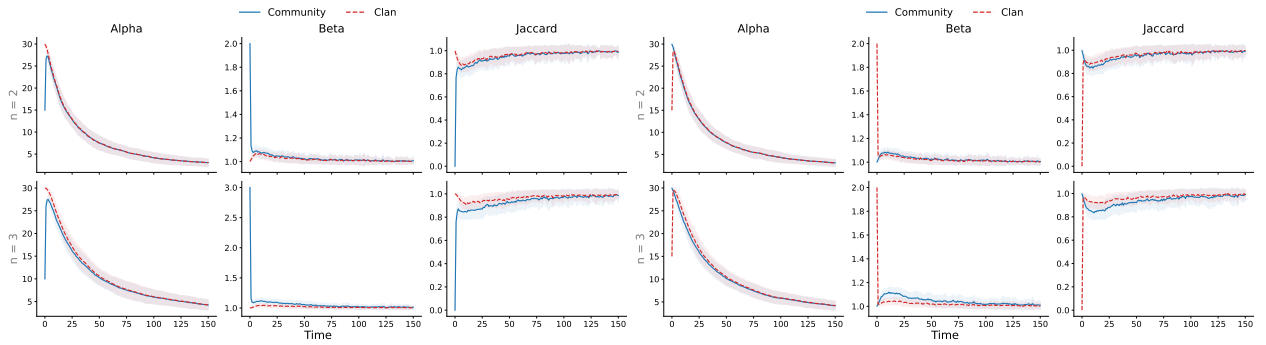
(a) Community initialization Parallel

(b) Clan initialization Parallel



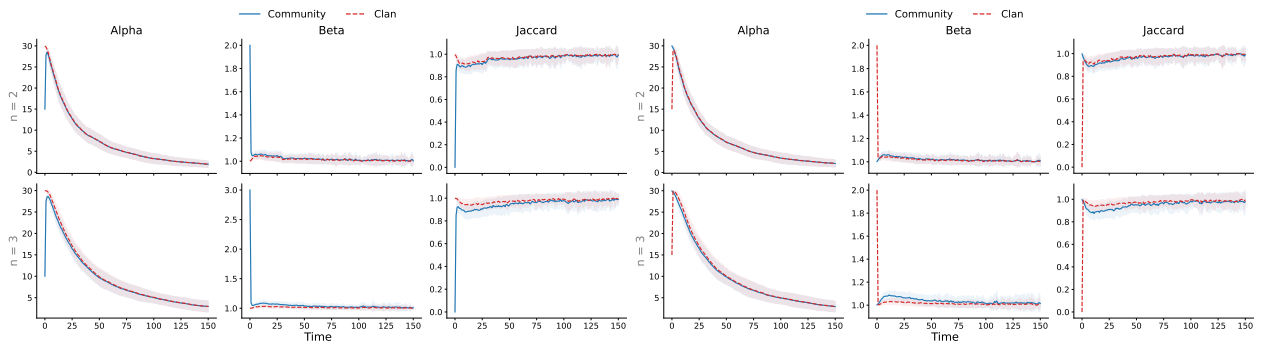
(c) Community initialization Dual Cross 33

(d) Clan initialization Dual Cross 33



(e) Community initialization Dual Cross 66

(f) Clan initialization Dual Cross 66



(g) Community initialization Dual Cross 100

(h) Clan initialization Dual Cross 100

Figure 4: Comparison diversity metrics community and clan level

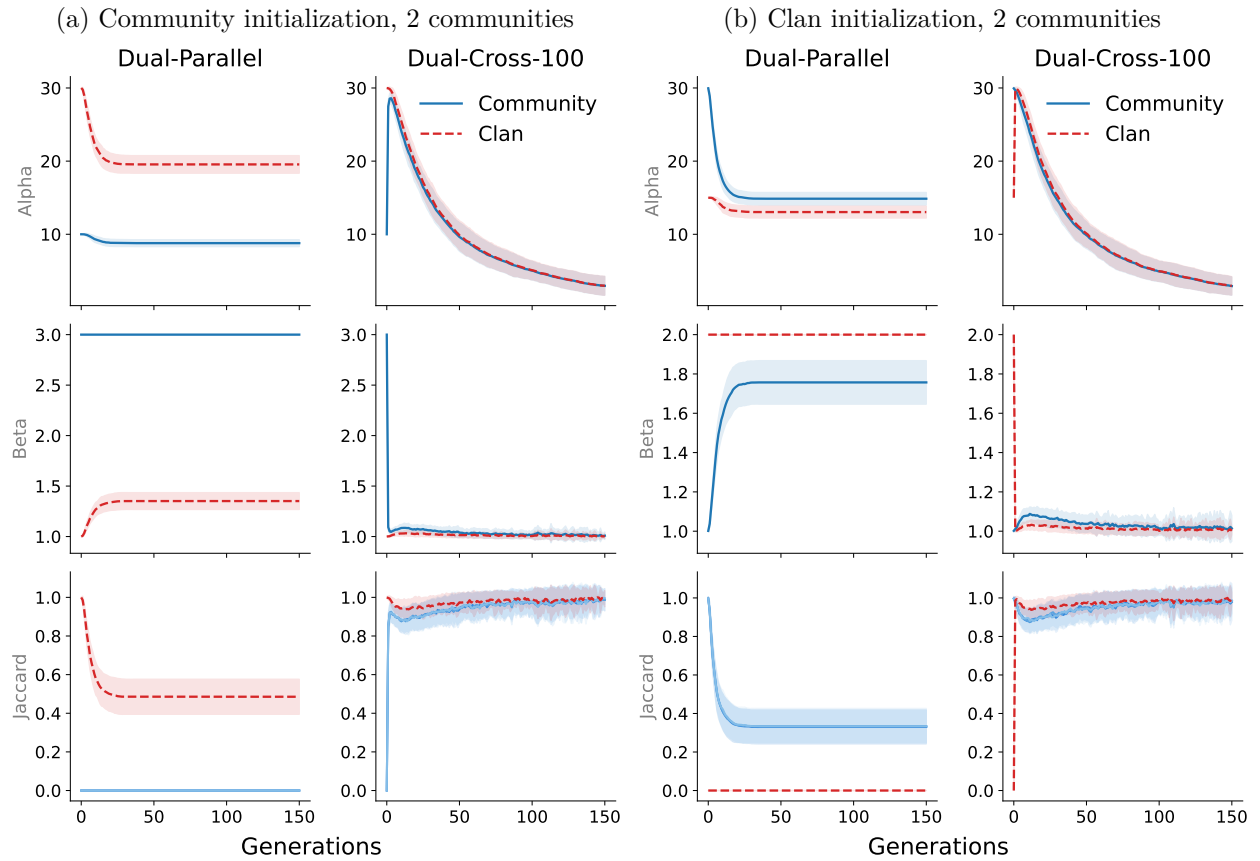
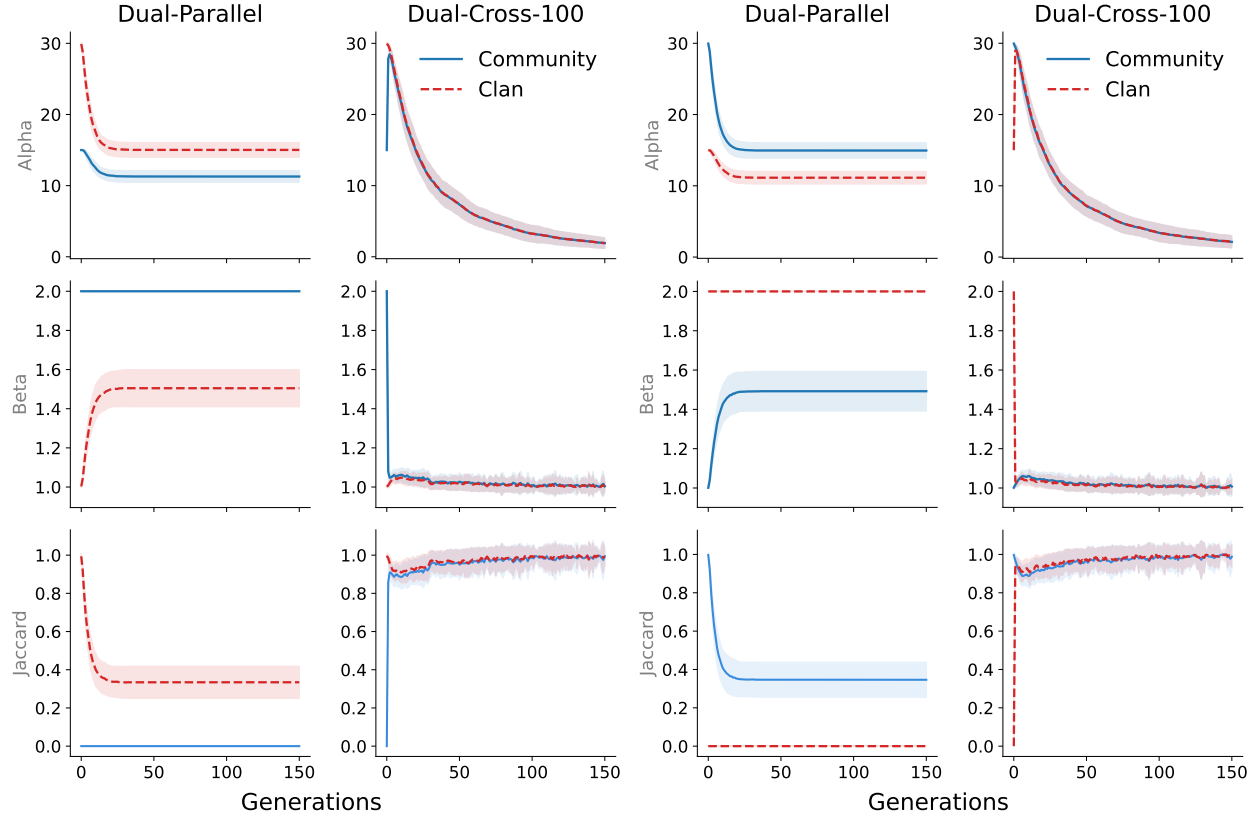


Figure 5: Comparison diversity metrics community and clan  $y$  for parallel and cross for 2 and 3 communities. Alternative visualization<sub>8</sub>

## 4 Key points

1. Although mix configurations lead to similar results, when they interact with the number of communities they do not always fall into homogeneization
2. We confirm that initialization does not affect the main result when we have cross transmission and that its structure is maintained when we have parallel transmission.